Building RAG Applications
A Practical Guide

A Technical Reference Document

Version 1.0 - 2025

1. Introduction to RAG

Retrieval-Augmented Generation (RAG) is an AI architecture pattern that enhances large language model (LLM) responses by grounding them in external knowledge sources. Instead of relying solely on the model's training data, RAG systems retrieve relevant documents at query time and include them as context for the model.

The key advantage of RAG over fine-tuning is that knowledge can be updated without retraining the model. This makes RAG ideal for applications that need access to frequently changing information, proprietary documents, or domain-specific knowledge bases.

A typical RAG pipeline consists of three stages: (1) Document ingestion - where source documents are processed, chunked, and indexed. (2) Retrieval - where relevant chunks are found based on the user's query. (3) Generation - where the LLM produces a response using both the query and retrieved context.

2. Document Processing Pipeline

The document processing pipeline transforms raw documents (PDFs, web pages, markdown files) into indexed chunks suitable for retrieval. This is typically an offline process that runs when new documents are added to the knowledge base.

Text extraction is the first step. For PDFs, libraries like pypdf or pdfplumber extract text content page by page. For web pages, HTML is parsed and boilerplate content (navigation, footers) is removed. Markdown files can be processed directly with their structure preserved.

Chunking strategies significantly impact retrieval quality. Common approaches include: fixed-size chunking (split every N tokens), semantic chunking (split at paragraph or section boundaries), and recursive chunking (hierarchically split large sections into smaller ones). The optimal chunk size typically ranges from 200 to 1000 tokens.

Each chunk is then enriched with metadata including the source document name, page number, section heading, and position within the document. This metadata enables citation generation and helps the LLM understand the context of retrieved passages.

3. Retrieval Strategies

The retrieval component is responsible for finding the most relevant chunks given a user query. The quality of retrieval directly determines the quality of the final response, making this the most critical component of a RAG system.

Vector similarity search is the most common retrieval method. Documents are embedded into high-dimensional vectors using embedding models (like text-embedding-3-small or all-MiniLM-L6-v2), stored in a vector database, and retrieved by cosine similarity to the query embedding.

Keyword-based retrieval using BM25 or TF-IDF provides complementary signals to vector search. Hybrid retrieval combines both approaches, often using Reciprocal Rank Fusion (RRF) to merge results. This typically outperforms either method alone.

Advanced retrieval techniques include: query expansion (reformulating the query for better recall), re-ranking (using a cross-encoder to reorder initial results), and multi-step retrieval (iteratively refining the search based on initial findings).

4. Context Window Management

Modern LLMs have context windows ranging from 8K to 200K tokens. Effectively managing this context window is crucial for RAG performance. Including too little context may miss relevant information, while including too much can dilute important details or exceed token limits.

A recommended approach is to retrieve 5-10 chunks initially, then select the top 3-5 based on relevance scoring. The selected chunks should be ordered by their position in the source document to maintain logical flow.

Context formatting matters. Each retrieved chunk should be clearly delineated with source attribution (document name, page number). A common format is: '[Source: Document Name, Page X]\n<chunk content>\n---'. This helps the LLM distinguish between different sources and generate accurate citations.

For long conversations, implement a sliding window strategy: maintain the system prompt and recent messages in full, but summarize older conversation turns. This preserves conversation continuity while leaving room for retrieved context.

5. Citation and Source Attribution

Citation generation is a distinguishing feature of well-built RAG systems. When the LLM generates a response based on retrieved documents, it should indicate which sources informed each claim, allowing users to verify the information.

Implementation approaches for citations include: (1) Prompt-based - instruct the LLM to include inline citations like [1], [2] referencing numbered sources. (2) Post-processing - match response sentences to source chunks using similarity scoring. (3) Structured output - use function calling to emit citation objects alongside the response text.

The structured output approach provides the most reliable citations. Define a tool or function that the model calls to emit citations with fields like document_name, page_range, and relevant_quote. The frontend can then render these as interactive reference cards.

Best practices for citation UX: display citations inline near the relevant text, provide hover previews showing the source passage, and allow users to click through to the full source document at the specific page or section.

6. Evaluation and Quality Metrics

Evaluating RAG systems requires measuring both retrieval quality and generation quality. Key metrics include: Retrieval Precision (what fraction of retrieved chunks are relevant), Retrieval Recall (what fraction of relevant chunks are retrieved), and Answer Correctness (does the generated answer match the ground truth).

Faithfulness measures whether the generated response is supported by the retrieved context. A faithful response makes no claims that cannot be traced back to the provided sources. This is critical for preventing hallucination in RAG systems.

Automated evaluation frameworks like RAGAS provide standardized metrics. The four core RAGAS metrics are: faithfulness, answer relevancy, context precision, and context recall. Running these on a test set of questions with known answers provides quantitative quality assessment.

Human evaluation remains important for subjective quality dimensions like readability, helpfulness, and citation accuracy. A combination of automated metrics for regression testing and periodic human evaluation for quality auditing provides comprehensive coverage.

7. Production Considerations

Deploying RAG systems to production introduces challenges around latency, cost, and reliability. Retrieval adds 100-500ms to response time depending on the vector database and number of chunks retrieved. Caching frequently asked questions and their retrieved contexts can significantly reduce latency.

Cost optimization strategies include: using smaller embedding models for initial retrieval, applying a re-ranker only to top candidates, caching embeddings for repeated queries, and implementing tiered storage (hot/warm/cold) for the knowledge base based on access patterns.

Error handling should account for: empty retrieval results (fall back to general knowledge), timeout on vector database queries (return cached or degraded response), and malformed documents (skip and log for manual review). The system should never fail silently.

Monitoring and observability are essential. Track metrics including: retrieval latency, number of chunks retrieved per query, token usage per request, citation accuracy over time, and user feedback signals (thumbs up/down). These metrics guide iterative improvements to chunking, retrieval, and prompting strategies.